Fondements de l'intelligence artificielle

Unité 4: problème de satisfaction de contraintes
Partie 1: Modélisation

Redoaune Ezzahir
red.ezzahir@gmail.com

Plan

- Définition formelle de CSP
- Modélisation de problèmes en CSP
- Résolution de CSP
  - Backtracking
  - Intelligent Backtracking
  - Heuristique de sélection de variables et de valeurs

Exemple introductif (sudoku)

- Les cases inconnues sont des variables
- Domaine :

  1 2 3 4 5 6 7 8 9
- Contraintes :
  - Lignes : ~~2 3 4 6 7~~
  - Colonnes : ~~1 2 3 5 8~~
- Réduction du domaine: 9

5		2			3		9	
8	3	6	2	1	9	4	7	5
	1		6				8	3
	7		1	3				8
9					2			1
1		8	9			5	3	
3			4	6			5	2
		5		9			1	
4		7		2		3	6	?

Exemple introductif (sudoku)

- Domaine : 1 2 3 4 5 6 7 8 9
- Contraintes :
  - Lignes : ~~2 3 5 9~~
  - Colonnes : ~~1 2 3 5 8 9~~
- Réduction du domaine: 4 6 7
- Résonnement local : ~~4 7~~
  - Valeur 6

5		2			3		9	**?**
8	3	6	2	1	9	4	7	5
	1		6				8	3
	7		1	3				8
9					2			1
1		8	9			5	3	
3			4	6			5	2
		5		9			1	
4		7		2		3	6	**9**

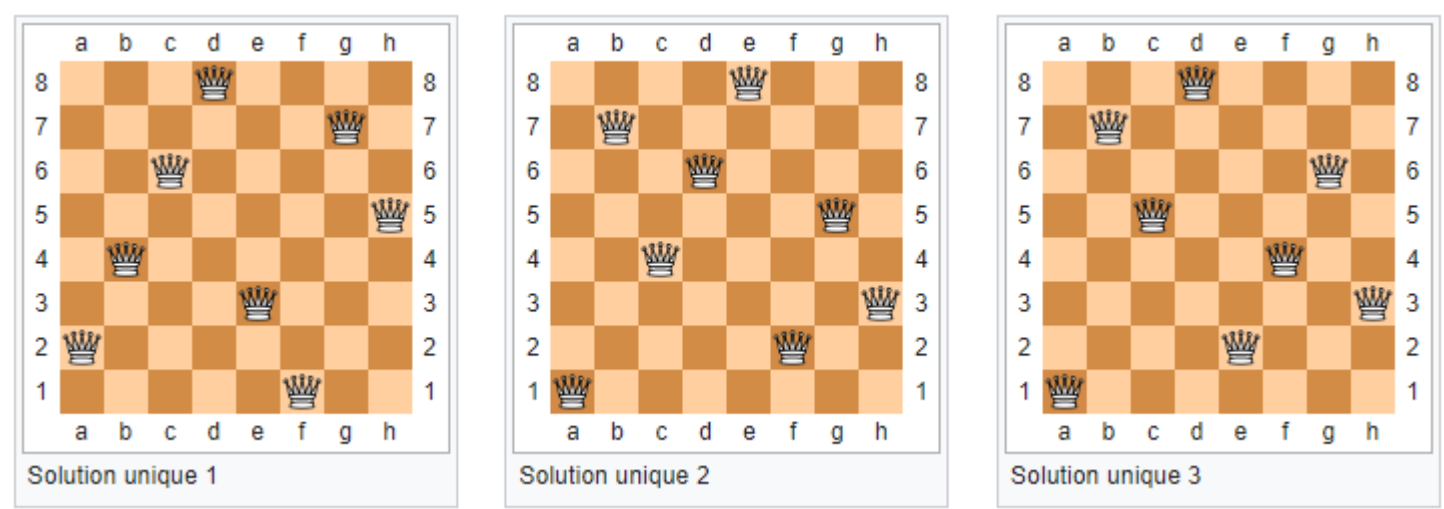
Exemple introductif (sudoku)

- Transmission de déductions aux autre régions
  - Propagation de contraintes
- Trois principes au cours du raisonnement par contraintes:
  - Raisonnement par élimination, réduction de domaine
  - Raisonnement local
  - Transmission des déductions aux autre régions (propagation)

5	~~1~~	2	~~1~~	~~1~~	3	**1**	9	**6**
8	3	6	2	1	9	4	7	5
	1		6				8	3
	7		1	3				8
9					2			1
1		8	9			5	3	
3			4	6			5	2
		5		9			1	
4		7		2		3	6	**9**

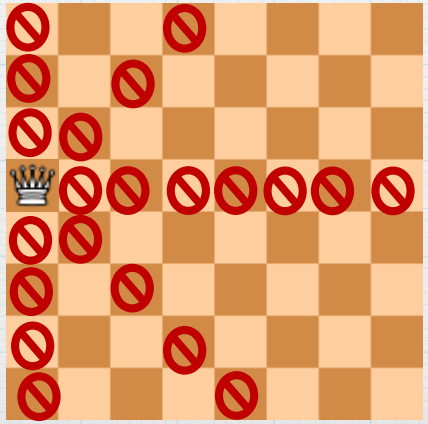
Exemple Motivant : 8-Reines

Placer 8 reines sur un échiquier de sorte qu'aucune n'attaque une autre.



Un algorithme de recherche exhaustive qui place une seule reines par rangée, va générer et tester $8^8 = 2^{24} = 16\ 777\ 216$ placements possibles.

Propagation de contraintes



Réduction dramatique de l'ensemble de possibilité

CSP comme problème de recherche

Un CSP peut facilement exprimée en problème de recherche standard:

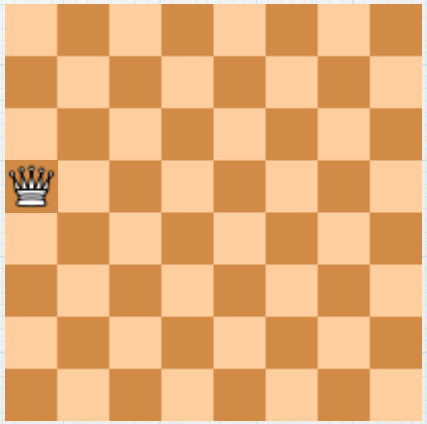
- Etat initial: l'affectation vide {}.
- Fonction successeur: Attribuer une valeur à une variable non affectée à condition qu'il n'y ait pas de conflit (tous les contraintes sont satisfaites).
- Teste de but: l'affectation courante est totale.
- Cout de chemin: un coût constant pour chaque pas.

- Solution se trouve toujours en profondeur n, pour n variables.

Quoi de plus pour 8 reines?

* Pas seulement une fonction successeur et test de but mais aussi:
  * Un moyen pour propager les contraintes

    imposées par une reine sur les autres
  * Un test d'échec précoce
  * Une représentation explicite des contraintes ➔ algorithmes de manipulation de contraintes



Définition d'un réseau de contraintes (CN)

Un CSP ou un réseau de contraintes est un triplet (X, D, C) où

- $X = \{X_1, .., X_n\}$ est un ensemble fini de n variables
- $D = \{D_1, .., D_n\}$, domaines finis où $X_i \in D_i$;
- $C = \{C_1, .., C_m\}$, m contraintes sur les variables où chacune définit un prédicat, qui est une relation sur un sous-ensemble particulier de variables (X)
  - i.e. C_i implique les variables $\{X_{i1}, X_{i2}, \ldots X_{ik}\}$ et définie la relation $R_i \subseteq D_{i1} \times D_{i2} \times \ldots D_{ik}$

Exemple

- *Soit P= (X, D,C)*
  - $X = \{ X_1, X_2, X_3, X_4\}$
  - $D = \{ D_1, D_2, D_3, D_4\}$, $D_1= D_2= D_3= D_4= \{0, 1\}$
  - $C = \{ C_1, C_2, C_3\}$,
  - $C_1 : R_1$, $C_2 : R_2$, $C_3 : R_3$
- $R_1 = X_1 \vee X_2 \vee X_3$
- $R_2 = X_1 \wedge X_4$
- $R_3 = X_3 \vee X_4$

R_1

X1	X2	X3
1	0	0
0	1	0
0	0	1
1	1	0
1	0	1
1	1	0
0	1	1
0	1	1
1	0	1
1	1	1

R_2

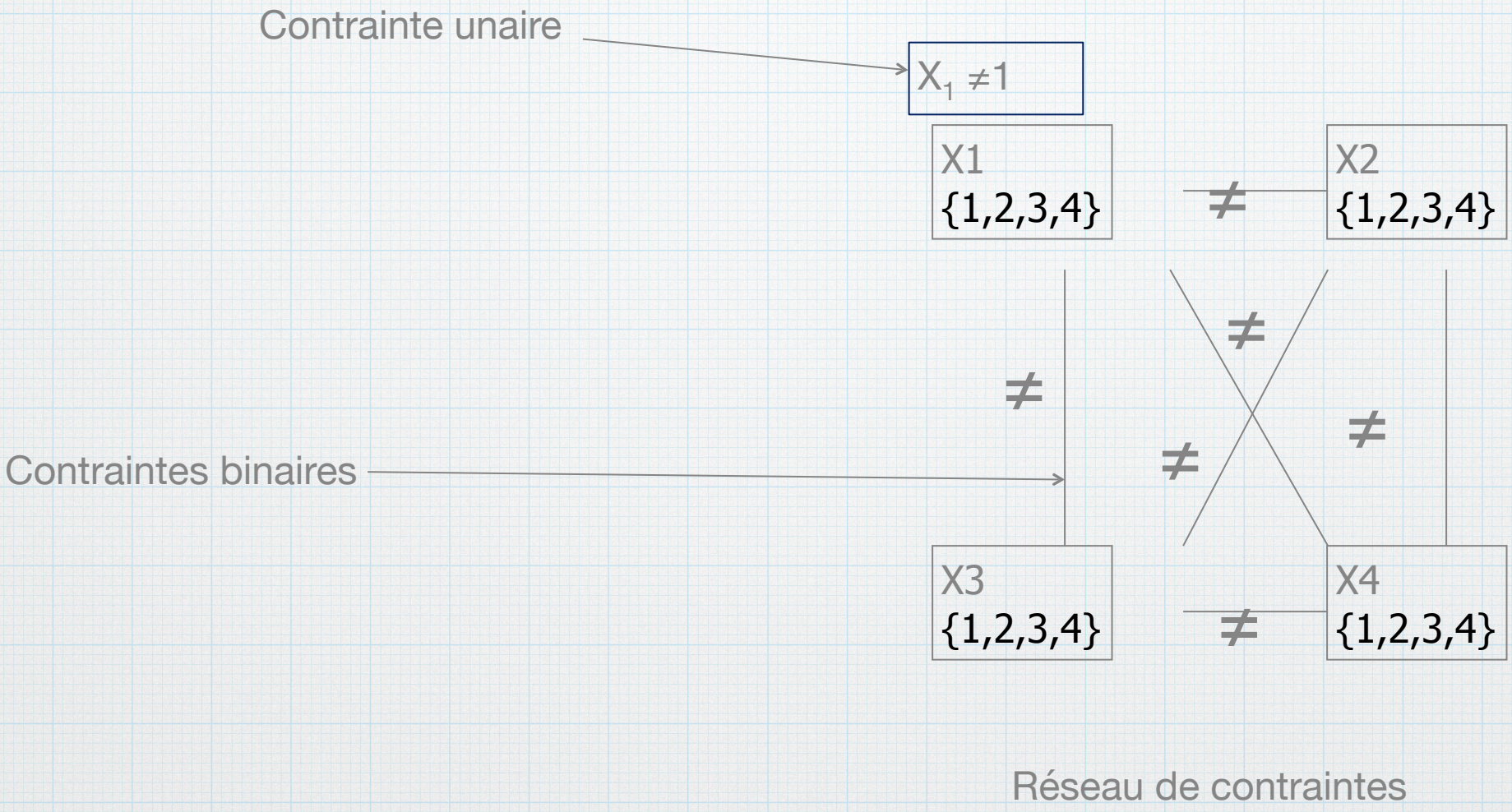
X1	x4
1	1

R_3

x3	x4
1	0
0	1
1	1

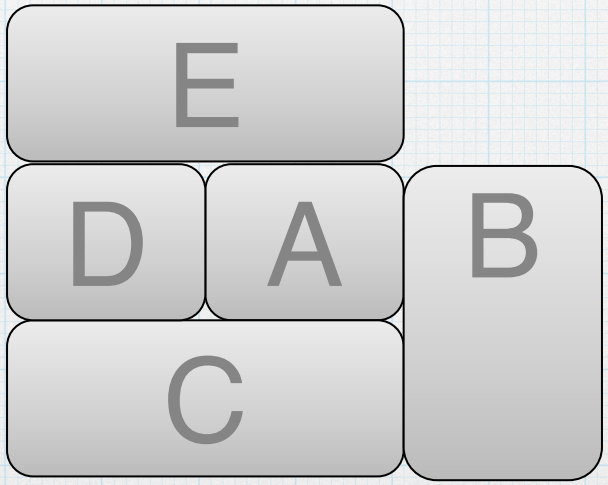
Solution = $\{ X_1=1,\ X_2 =0,\ X_3=0,\ X_4=1\}$

Contraintes unaires et binaires les plus courantes

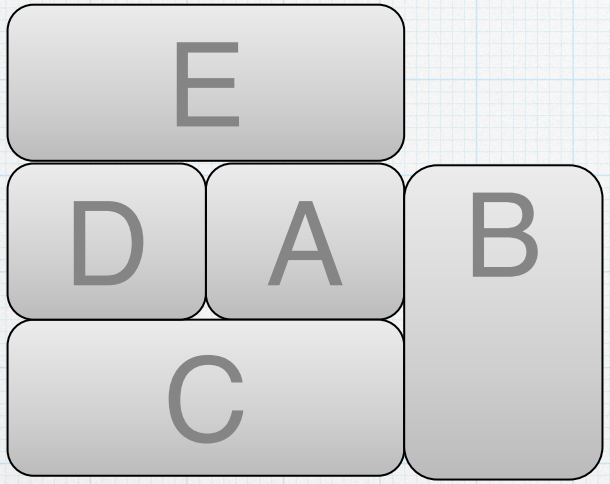


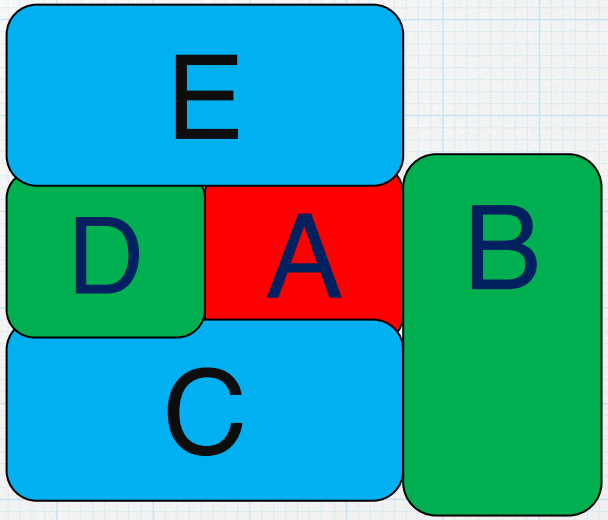
Exemple: coloration de la carte (1)

* Colorier la carte suivante à l'aide de trois couleurs (rouge, vert, bleu) de façon que deux régions adjacentes n’ont pas la même couleur



Exemple: coloration de la carte (2)





- Variables: A, B, C, D, E tous ont le domaine RGB
- Domaines: RGB = {red, green, blue}
- Contraintes: A≠B, A ≠ C, A ≠ E, A ≠ D,

  B ≠ C, C ≠ D, D ≠ E
- Une solution:

  {A=red, B=green, C=blue, D=green, E=blue}

Problèmes du monde réel

* Les CSP sont un bon modèle pour de nombreux problèmes pratiques qui se posent dans le monde réel.

- ordonnancement
- raisonnement temporel
- La conception des bâtiments
- planification
- Optimisation / satisfaction
- vision

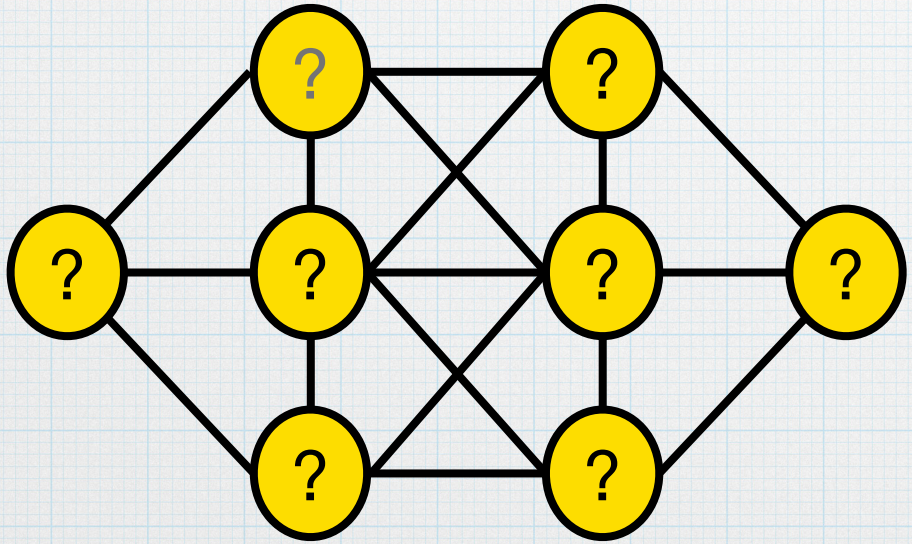
Disposition de graphe
La gestion du réseau
Traitement du langage naturel
Biologie / génomique moléculaire
conception VLSI
…

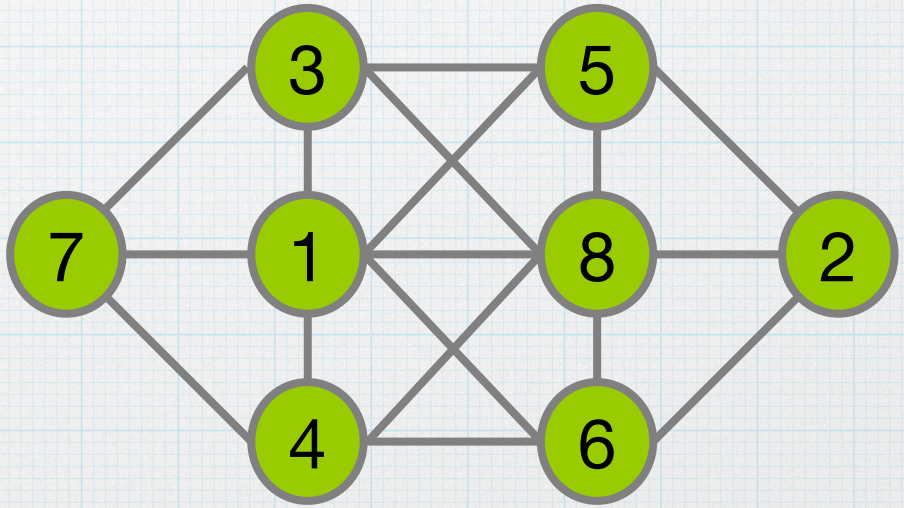
Deux niveaux en CSP

- Représentation du problème
  - Modélisation des contraintes selon un formalisme donné (>, <, =, ≠, …)
- Résolution du problème
  - Utiliser un solveur de contraintes.

Exercice (modéliser en CSP)

Exemple (graphe de contraintes):
Placer les numéros 1 à 8 sur les nœuds, où chaque chiffre apparaisse exactement une fois et pas de nœuds connectés ont des numéros consécutifs





Exemples de modélisation
(Faire un plan de maison)

Appartement de 5 pièces (Chambre, Bain, Cuisine, Salon, Salle-à-Manger) sur un emplacement à 6 pièces

- ✓ Cuisine et salle à manger communiquent
- ✓ SdeB et cuisine ont un mur en commun
- ✓ SdeB éloignée de salon
- ✓ Salon et SàM sont une même pièce

nord-ouest	nord	Nord Est
sud-ouest	sud	sud-est

Modèle CSP pour le plan de maison (1)

- Variables :
  - C (chambre),
  - B (salle de bain),
  - K (cuisine),
  - S (salon),
  - M (salle à manger)

no	n	ne
so	s	se

- Domaines : {no,n,ne,so,s,se}

Modèle CSP pour le plan de maison (2)

no	n	ne
so	s	se

- Contraintes :
  - $X \neq Y$, $X = Y$
  - **`A-cote`**(X,Y) = {

(no,n),(no,so),(n,no),(n,ne),(n,s), (ne,n),(ne,se),(so,no), (so,s),(s,so), (s,n),(s,se),(se,s),(se,ne) }

  - **`Loin`**(X,Y) = {

(no,ne),(no,s),(no,se),(n,so),(n,se), (ne,no),(ne,so), (ne,s),(so,n),(so,ne), (so,se),(s,no),(s,ne),(se,no),(se,n), (se,so) }

Cryptarithmetic

- Trouvez des chiffres différents tel que

  - Et T et F sont non nuls

```
  r3 r2 r1
     TWO
+    TWO
 ---------
   = FOUR
```

- Variables : $X = \{F, O, U, R, T, W, r1, r2, r3\}$
- Les Domaines : $Di = \{0, \ldots, 9\}$
- Contraintes:
  - Alldiff (F,T,U,W,R,O)
  - $O + O = R + 10 \cdot r1$
  - $r1 + W + W = U + 10 \cdot r2$
  - $r2 + T + T = O + 10 \cdot r3$
  - $r3 = F$,
  - $T \neq 0$, $F \neq 0$

Exercice 1 : Retour de monnaie

* On s'intéresse à un distributeur automatique de boissons. L'utilisateur insère des pièces de monnaie pour un total de T centimes de Dirhams, puis il sélectionne une boisson, dont le prix est de P centimes de Dirhams (T et P étant des multiples de 10). Il s'agit alors de calculer la monnaie à rendre, sachant que le distributeur a en réserve Nb2 pièces de 2 Dh, Nb1 pièces de 1 Dh, Nb50 pièces de 50 centimes, Nb20 pièces de 20 centimes et Nb10 pièces de 10 centimes.

1. Modéliser ce problème sous forme de CSP?

2. Exprimer «on souhaite rendre le moindre de pièces possible »?

vous avez gagner 1 point de bonus pour lecture avant cours (mémorisez le titre du slide).

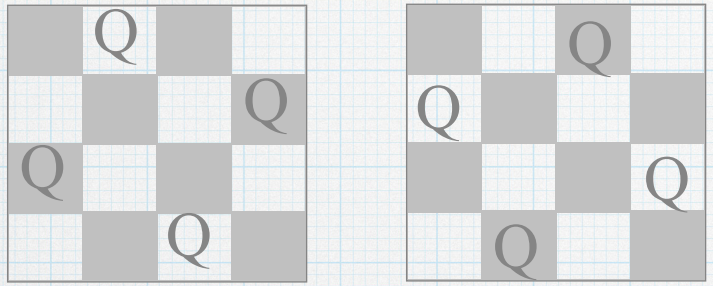
Modèle CSP pour « Retour de monnaie »

- Variables : X = {X2, X1, X50, X20, X10}
- Les domaines spécifient que la quantité de pièces retournées, pour un type de pièce donné, est comprise entre 0 et le nombre de pièces de ce type que l'on a en réserve : D(X2) = {0,1,...,Nb2} , D(X1) = {0,1,...,Nb1}, D(X50) = {0,1,...,Nb50} , D(X20) = {0,1,...,Nb20} , D(X10) = {0,1,...,Nb10}
- Les contraintes spécifient que la somme à retourner doit être égale à la somme insérée moins le prix à payer : C = { 200*X2 + 100*X1 + 50*X50 + 20*X20 + 10*X10 = T-P }

2. Ajouter et Minimiser une variable XR: {0, ..., Nb_PIECES_DISPONIBLE} / XR = ∑Xi

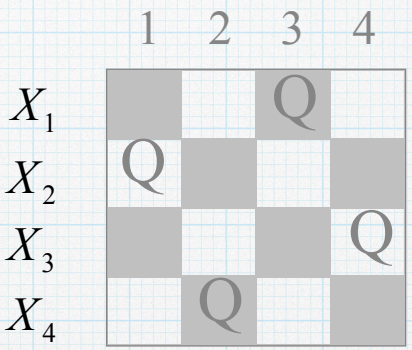
problème des 4-Reines

Placer 4 Reines sur un échiquier de 4x4 tq. deux Reines ne soit pas sur la même ligne, colonne, ou diagonale.



Formulation standard du problème:

- **Variables**: chaque ligne est une variable.



- **Domaines**: $D_i = \{1,2,3,4\}$.

- **Contraintes**: Il y a $\binom{4}{2}$ = 6 contraintes: $\forall i < j, |Xi - Xj| \neq j - i$

$R_{12} = \{(1,3)(1,4)(2,4)(3,1)(4,1)(4,2)\}$

$R_{13} = \{(1,2)(1,4)(2,1)(2,3)(3,2)(3,4)(4,1)(4,3)\}$

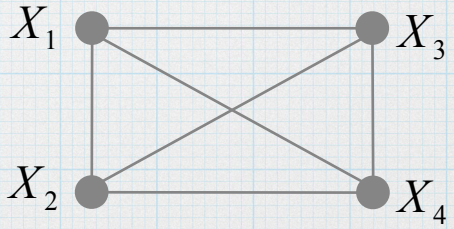
$R_{14} = \{(1,2)(1,3)(2,1)(2,3)(2,4)(3,1)(3,2)(3,4)(4,2)(4,3)\}$

$R_{23} = \{(1,3)(1,4)(2,4)(3,1)(4,1)(4,2)\}$

$R_{24} = \{(1,2)(1,4)(2,1)(2,3)(3,2)(3,4)(4,1)(4,3)\}$

$R_{34} = \{(1,3)(1,4)(2,4)(3,1)(4,1)(4,2)\}$

- **Graphe de contraintes** :



Résolution de CSP

- Énoncés possibles
  - Existe t-il une solution?
  - Trouver une solution
  - Trouver toutes les solutions
  - Trouver le nombre de solutions
  - Telle valeur figure-t-elle dans une solution?
  - Trouver toutes les valeurs possible pour une variable
  - Trouver une valeur figurant dans toute les solutions
  - Trouver une solution optimale
  - …

Le problème de décision est NP-Complet

Résolution de CSP

- Résolution : affectation de valeurs aux variables de telle sorte que toutes les contraintes soient satisfaites
- affectation partielle : instanciation des variables : $\{x_1 = v_1, \ldots x_p = v_p\}$
- affectation totale : affectation de toutes les variables $\{x_1 = v_1, \ldots x_n = v_n\}$
- affectation consistante : affectation qui ne viole aucune contrainte
- affectation inconsistante : affectation qui viole au moins une contrainte
- solution : affectation totale et consistante

Méthodes de Résolution de CSP

- Générer et Tester
  - Principe : Générer chaque affectation possible des variables et tester si les contraintes sont satisfaites
  - Possibilités exponentielles: $O(d^n)$
  - Simple mais extrêmement gaspillant!
- Retour arrière (Backtrack)
  - Principe: Teste les contraintes après chaque instanciation d'une variable et revenir en arrière si une violation a lieu.
  - Tente progressivement d'étendre une solution partielle
  - Des sous-arbres sont entièrement éliminés en une fois

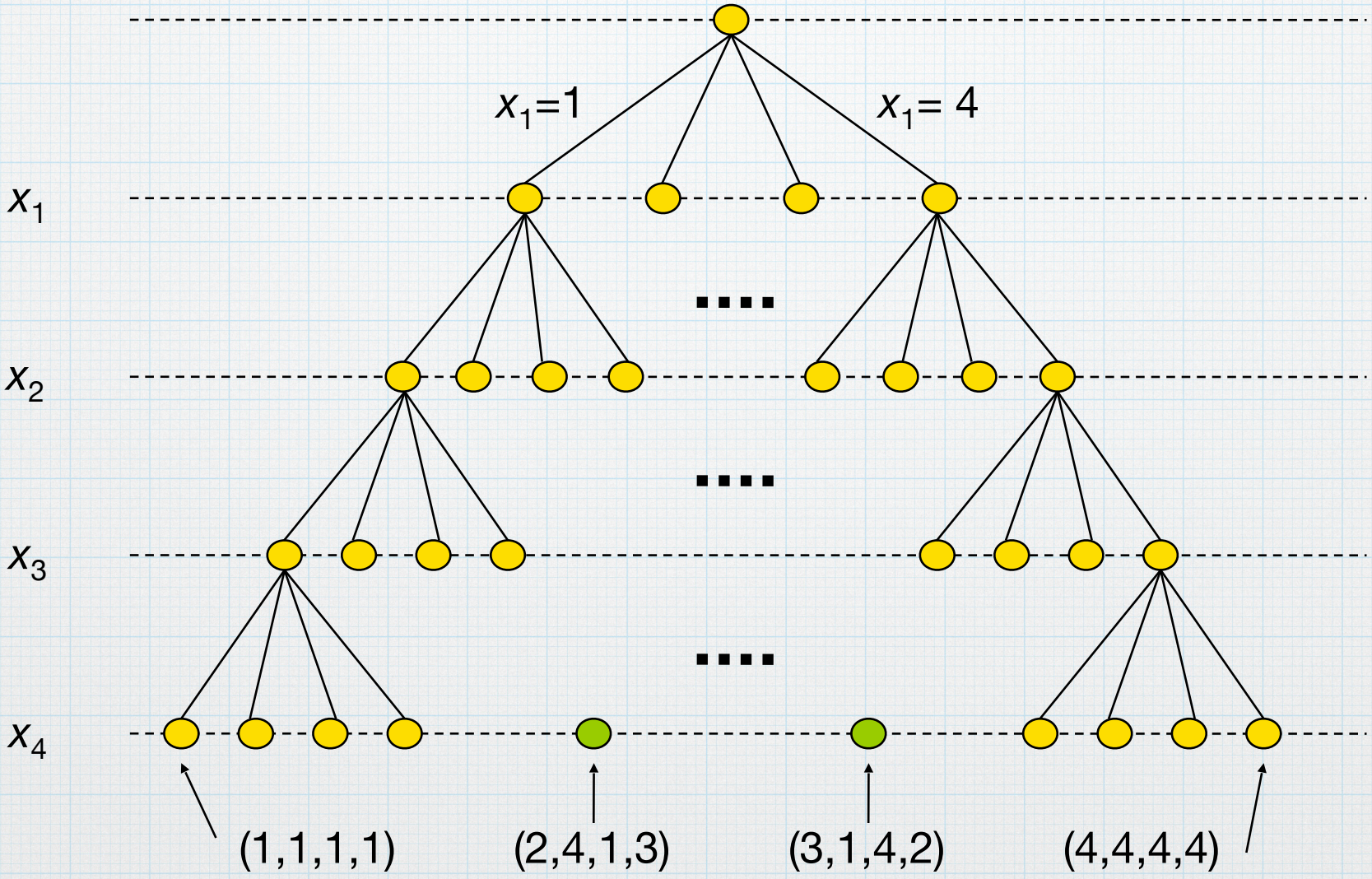
Algorithme Génère Et Teste : GET

```
fonction GET(A,(X,D,C,R)) : booléen
début
 si toutes les variables de X sont affectées alors
        si  A est consistante alors
                retourner VRAI
        sinon
                retourner FAUX
        finsi
 sinon
        choisir une variable Xi de X qui n'est pas encore affectée
        pour toute valeur Vi appartenant à Di faire
                si GET(A ∪ {(Xi ,Vi )}, (X,D,C,R)) = VRAI alors
                        retourner VRAI
                finsi
        fin pour
        retourner FAUX
  finsi
fin
```

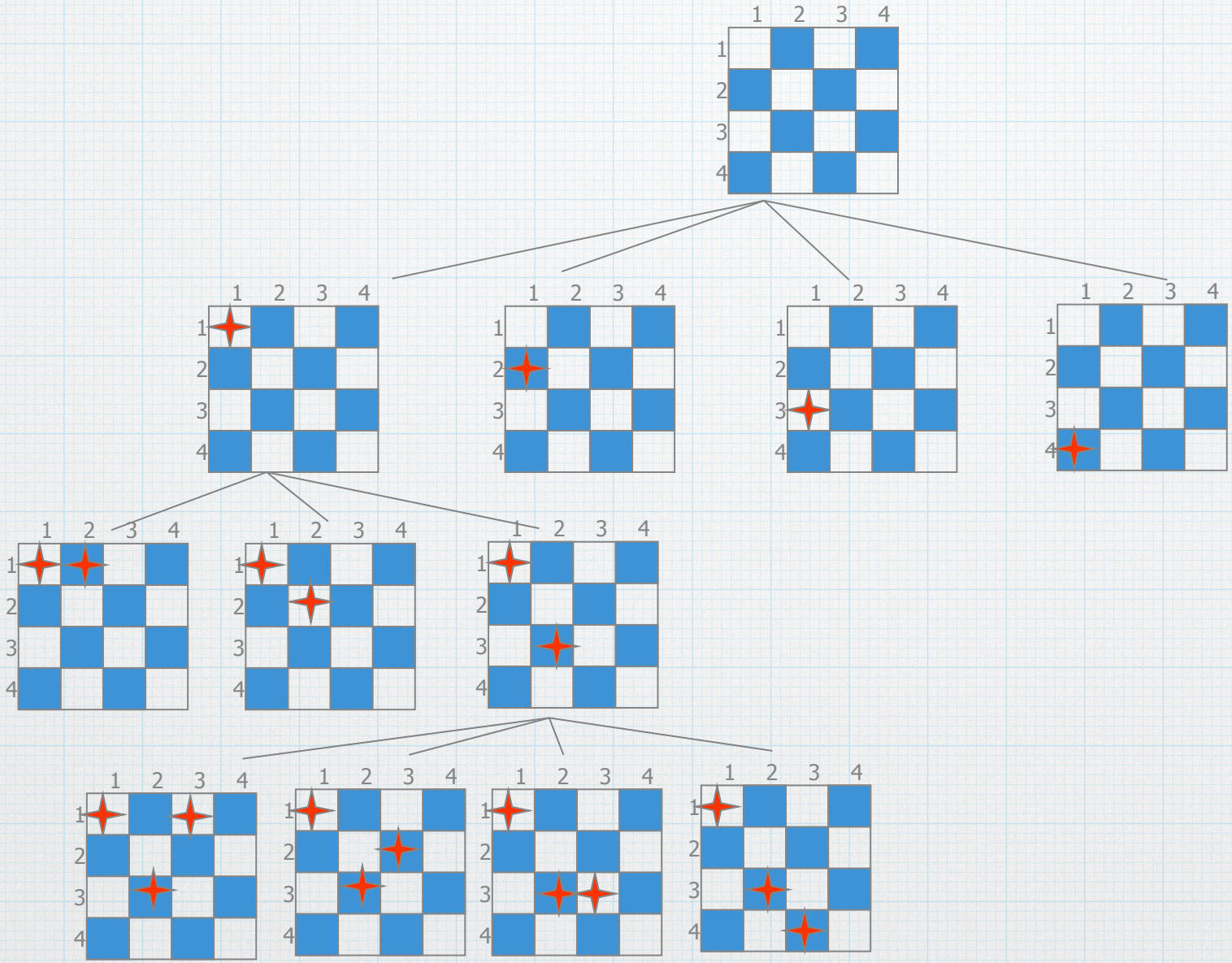
Backtracking

```
fonction BT(A,(X,D,C,R)) : booléen
début
    si  A est consistante alors
        si toutes les variables de X sont affectées alors
            retourner VRAI
    Sinon
        retourner FAUX
    finsi
    choisir une variable Xi de X qui n'est pas encore affectée
    pour toute valeur Vi appartenant à Di faire
        si BT(A ∪ {(Xi ,Vi )}, (X,D,C,R)) = VRAI alors
            retourner VRAI
        finsi
    fin pour
    retourner FAUX
fin
```

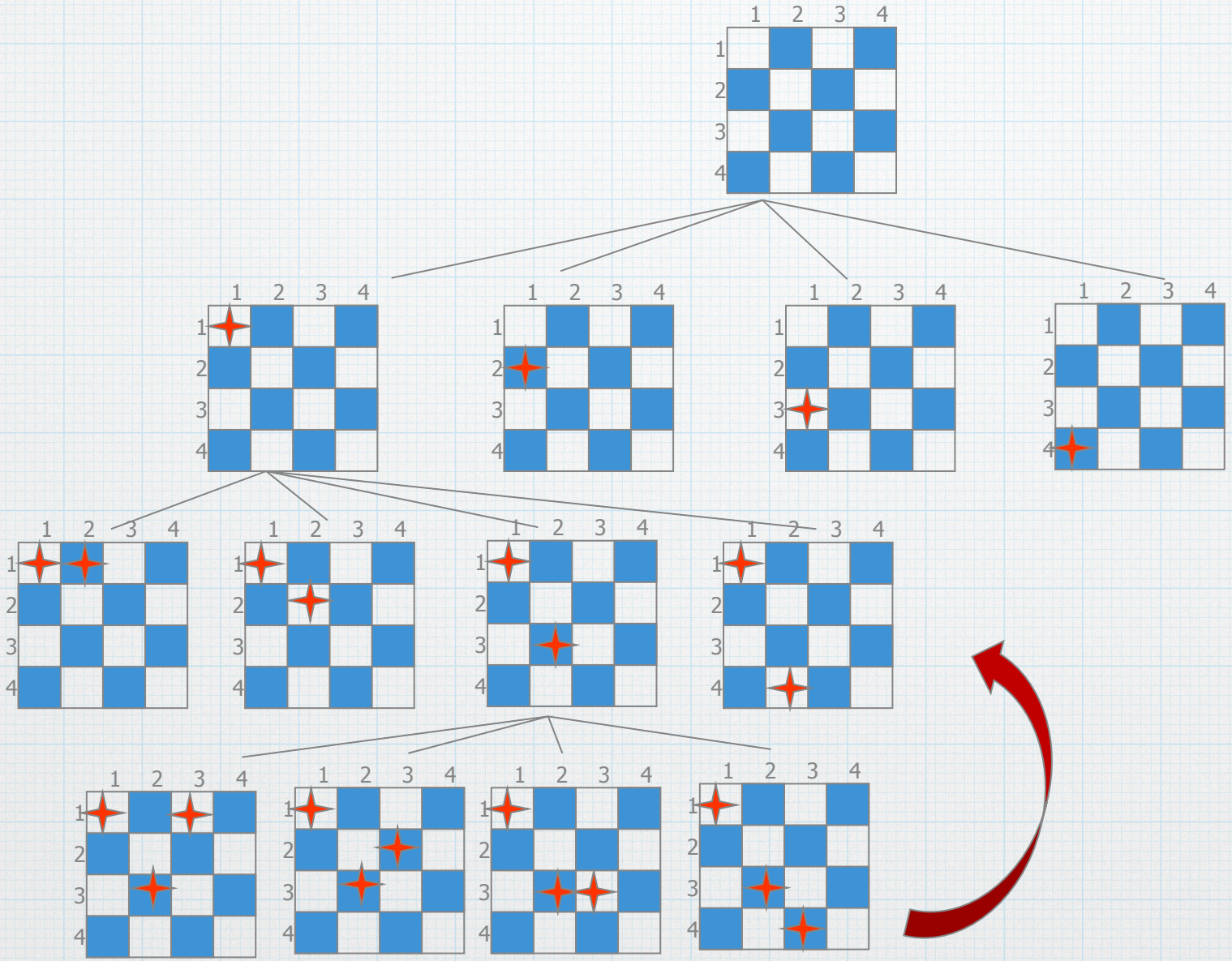
Arbre de recherche pour 4-Reines



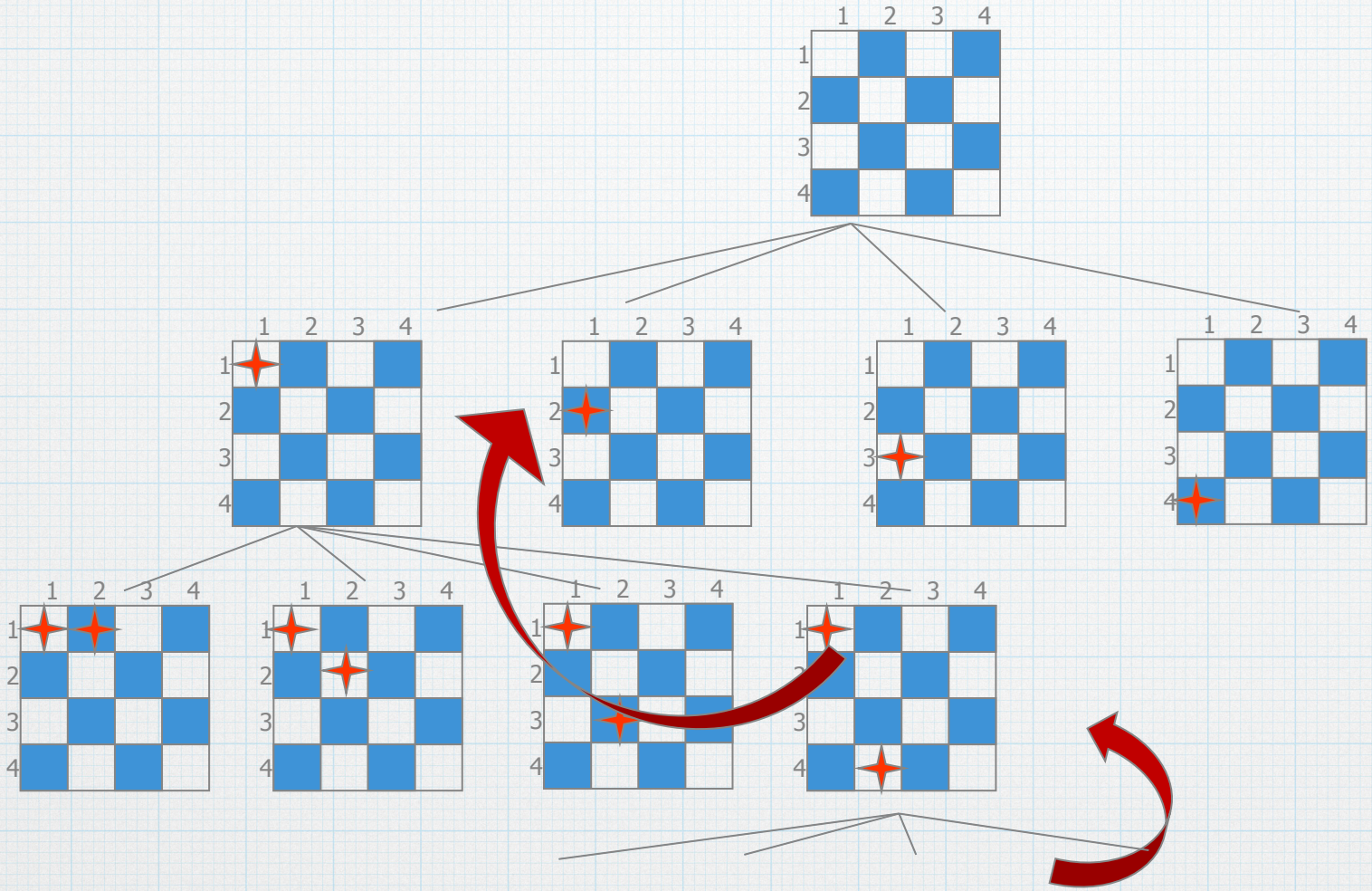
Backtracking search



Backtracking search

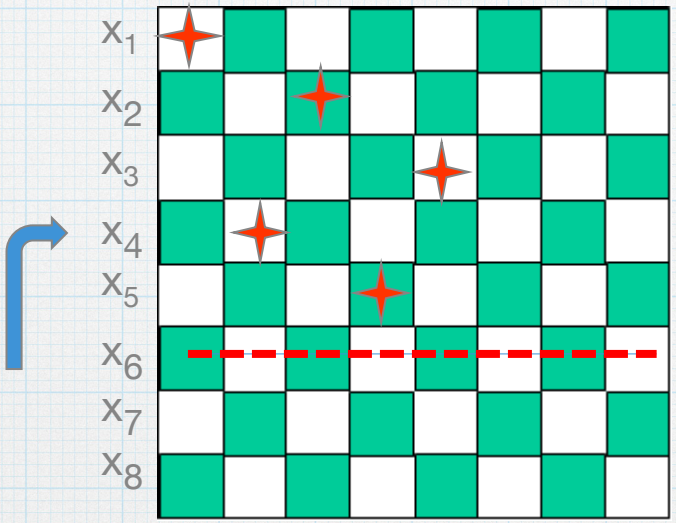


Backtracking search



Intelligent Backtracking (backjump)

Ensemble de conflit (conflict set)
$CS(x_k)$



{}
{1}
{1,2}
{1}
{1,2,4}
{1,2,3,4}

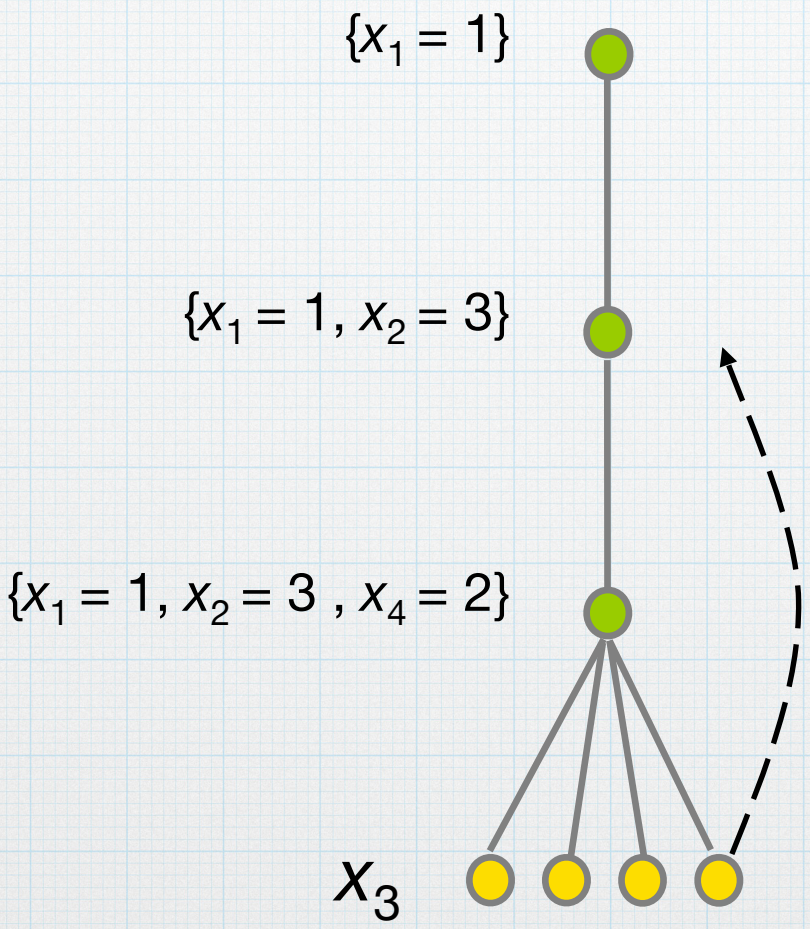
Retour arrière intelligent :

saut vers la dernière variable x_j dans l'ensemble de conflit $CS(x_k)$

mis-à-jour de l'ensemble de conflit :

$$CS(x_j) = CS(x_k) \cup CS(x_j) / \{x_j\}$$

Conflict-directed backjumping (CBJ)



	x_1	x_2	x_3	x_4
1	Q		x_1	
2			x_2	Q
3		Q	x_1	
4			x_2	

Heuristiques pour le Backtracking

- Ordre sur les variables (très important)
  - Quelle variable suivante à brancher
- Ordre sur les valeurs (peut être important)
  - étant donné le choix de la variable, quelle valeur à essayer

Ordonnancement de variable

* Heuristiques dépendantes de domaine
* Domaine indépendante heuristiques
* Ordre de variables statique fixé avant que la recherche commence
* Ordonnancement dynamique de variables choisi lors de la recherche

Heuristiques classiques (1)

* choix de valeurs
  * recherche d ’une solution : choix les moins contraints (fail-last)
  * recherche de toutes les solutions : choix les plus contraints (Fail-first)

* choix de variables (Fail-first)
  * en fonction de critères structurels : degré, tailles des domaines , satisfiabilité des contraintes...

Heuristiques classiques (2)

* lex: ordonne les variables selon l'ordre lexicographique (statique).
* Random sélectionne aléatoirement une variable non instanciée.
* min-domain ou dom: sélectionne la variable non instanciée qui a le plus petit domaine de valeurs.

Heuristiques classiques (2)

* degree ou deg l'heuristique du degré ordonne les variables selon leurs degrés décroissants

* dom/deg : il s'agit de combinaisons des heuristiques précédentes dans la quelle on sélectionne la variable dont le quotient de la taille du domaine sur le degré est minimal

Sudoku par un solveur CSP en python

```python
def sudoku(initValue):
    p = Problem()
    # Define a variable for each cell: 11,12,13...21,22,23...98,99
    for i in range(1, 10) :
        p.addVariables(range(i*10+1, i*10+10), range(1, 10))
    # Each row has different values
    for i in range(1, 10) :
        p.addConstraint(AllDifferentConstraint(), range(i*10+1, i*10+10))
    # Each colum has different values
    for i in range(1, 10) :
        p.addConstraint(AllDifferentConstraint(), range(10+i, 100+i, 10))
    # Each 3x3 box has different values
    p.addConstraint(AllDifferentConstraint(), [11,12,13,21,22,23,31,32,33])
    p.addConstraint(AllDifferentConstraint(), [41,42,43,51,52,53,61,62,63])
    p.addConstraint(AllDifferentConstraint(), [71,72,73,81,82,83,91,92,93])

    p.addConstraint(AllDifferentConstraint(), [14,15,16,24,25,26,34,35,36])
    p.addConstraint(AllDifferentConstraint(), [44,45,46,54,55,56,64,65,66])
    p.addConstraint(AllDifferentConstraint(), [74,75,76,84,85,86,94,95,96])

    p.addConstraint(AllDifferentConstraint(), [17,18,19,27,28,29,37,38,39])
    p.addConstraint(AllDifferentConstraint(), [47,48,49,57,58,59,67,68,69])
    p.addConstraint(AllDifferentConstraint(), [77,78,79,87,88,89,97,98,99])

    # add unary constraints for cells with initial non-zero values
    for i in range(1, 10) :
        for j in range(1, 10):
            value = initValue[i-1][j-1]
            if value:
                p.addConstraint(lambda var, val=value: var == val, (i*10+j,))
    return p.getSolution()
```

```python
# Sample problems
easy = [
  [0,9,0,7,0,0,8,6,0],
  [0,3,1,0,0,5,0,2,0],
  [8,0,6,0,0,0,0,0,0],
  [0,0,7,0,5,0,0,0,6],
  [0,0,0,3,0,7,0,0,0],
  [5,0,0,0,1,0,7,0,0],
  [0,0,0,0,0,0,1,0,9],
  [0,2,0,6,0,0,0,5,0],
  [0,5,4,0,0,8,0,7,0]]

hard = [
  [0,0,3,0,0,0,4,0,0],
  [0,0,0,0,7,0,0,0,0],
  [5,0,0,4,0,6,0,0,2],
  [0,0,4,0,0,0,8,0,0],
  [0,9,0,0,3,0,0,2,0],
  [0,0,7,0,0,0,5,0,0],
  [6,0,0,5,0,2,0,0,1],
  [0,0,0,0,9,0,0,0,0],
  [0,0,9,0,0,0,3,0,0]]

very_hard = [
  [0,0,0,0,0,0,0,0,0],
  [0,0,9,0,6,0,3,0,0],
  [0,7,0,3,0,4,0,9,0],
  [0,0,7,2,0,8,6,0,0],
  [0,4,0,0,0,0,0,7,0],
  [0,0,2,1,0,6,5,0,0],
  [0,1,0,9,0,5,0,4,0],
  [0,0,8,0,2,0,7,0,0],
  [0,0,0,0,0,0,0,0,0]]
```

À suivre...

* Recherche Locale

* Propagation de contraintes
  * La consistance d’arc